

Hamid & Ali — Training Sessions

This file covers the full backend of Mystery Room, split by ownership:

- **Hamid Al Haj** — Shared backend structure + first mystery (case 1).
- **Ali Mikdad** — Second mystery (case 2) + validation behavior + tests.

The shared controller file `server/controllers/mysteryController.js` is presented from both angles: Hamid explains the structure and the read endpoints, Ali explains validation, status codes, hint budget, and the public response builder.

PART 1 — HAMID AL HAJ

Hamid Al Haj — Shared Backend Structure + First Mystery

Session 1 — Node.js and Express Foundations

Learning Objectives

By the end of this session, the team should understand:

- What Node.js is and why it exists.
- What Express adds.
- How the project's server is structured.

1. The Problem

The game rules — current stage, hints used, which answer is correct — must not live in the browser. A player could inspect the client and cheat. The server must own the rules.

But JavaScript was written for the browser. How do we run it on a server?

2. Familiar Idea

Every `npm run dev` we ran before was itself a Node.js program. Node.js runs JavaScript outside the browser.

3. The New Concept

- **Node.js** is a **runtime**: a program that executes JavaScript.
- **Express** is a library that runs on Node. It maps HTTP requests to functions.

The project's server is small: one entry file, one app file, one router, one controller, one store.

4. Tiny Example

```
import express from "express";
const app = express();
app.get("/hello", (req, res) => res.json({ message: "hi" }));
app.listen(3001);
```

Three lines: create, route, listen.

5. Project Application

File: `server/index.js`

```
import app from "./app.js";
import dotenv from "dotenv";
dotenv.config();
const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`Mystery Room API running at http://localhost:${PORT}`);
});
```

File: `server/app.js`

```
import express from "express";
import mysteriesRouter from "../routes/mysteries.js";

const app = express();

app.use(express.json({ limit: "10kb" }));
app.use("/api/mysteries", mysteriesRouter);
app.get("/", (req, res) => {
  res.json({ message: "Mystery Room API is running." });
});
app.use((req, res) => {
  res.status(404).json({ error: "This API route does not exist." });
});
app.use((error, req, res, next) => {
  if (error.type === "entity.parse.failed") {
    return res
      .status(400)
      .json({ error: "Request body must contain valid JSON." });
  }
  if (error.type === "entity.too.large") {
    return res.status(413).json({ error: "Request body is too large." });
  }
  next(error);
});

export default app;
```

File: server/package.json

```
{
  "name": "mystery-room-b4f-server",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "test": "node --test"
  },
  "dependencies": {
    "dotenv": "^18.0.4",
    "express": "^4.19.2"
  }
}
```

6. Code Walkthrough

`type: "module"`

What it does: Tells Node to use ES Modules (`import` / `export`). **Why it exists:** The same module syntax the client uses. **What you see:** No difference at runtime.

`dotenv.config()`

What it does: Reads `.env` and adds variables to `process.env`. **Why it exists:** `PORT` can be overridden without editing code. **What you see:** The server starts on 3001 by default.

`app.use(express.json({ limit: "10kb" })))`

What it does: Parses incoming JSON bodies up to 10 kilobytes. **Why it exists:** The answer submission sends `{ answer: "brass" }`. **What you see:** `req.body.answer` is available in the controller.

The 404 handler

```
app.use((req, res) => {
  res.status(404).json({ error: "This API route does not exist." });
});
```

What it does: Catches any request that no route matched. **Why it exists:** The API should not return HTML for unknown paths. **What you see:** A JSON error.

The error handler

```
app.use((error, req, res, next) => {
  if (error.type === "entity.parse.failed") {
    return res
      .status(400)
      .json({ error: "Request body must contain valid JSON." });
  }
  if (error.type === "entity.too.large") {
    return res.status(413).json({ error: "Request body is too large." });
  }
  next(error);
});
```

What it does: Handles body-parser errors with clear messages. **Why it exists:** Without it, malformed JSON would crash or produce an unclear response. **What you see:** A clear 400 or 413 JSON error.

7. Common Mistakes

WRONG: Using `require` in an ES Modules project.

```
const express = require("express"); // wrong when type is module
```

CORRECT:

```
import express from "express";
```

WRONG: Forgetting `app.listen` .

Without it, nothing listens on the port.

WRONG: Putting all logic in `index.js` .

The project separates: `index.js` starts, `app.js` composes, `routes/` maps, `controllers/` handles.

8. Important Decisions

- **Why ES Modules?** Consistent with the client.
- **Why separate `index.js` and `app.js` ?** The app can be tested without starting a listener.
- **Why a 10kb limit?** Answers are small; large bodies are suspicious.
- **Why custom 404 and error handlers?** They return JSON, not HTML.

9. Project Flow

```
npm start in server/
  |
  ▼
Node runs index.js
  |
  ▼
dotenv.config reads .env
  |
  ▼
app.listen(3001) starts listening
  |
  ▼
Requests arrive at app.js
  |
  ▼
Middleware runs: express.json, then routes, then 404
```

10. Review Questions

1. What is Node.js?
2. What does Express add?

3. What does `express.json()` do?
4. Why does the project have a 404 handler?
5. Why is `app.listen` important?

11. Glossary

Term	Meaning
Runtime	A program that executes code.
Express	A Node library for HTTP servers.
Middleware	A function in the request-processing chain.
ES Modules	The <code>import</code> / <code>export</code> syntax.
<code>.env</code>	A file of environment variables.

12. What You Should Be Able to Explain Now

- What Node.js and Express are.
- How the server is structured.
- What each middleware does.
- Why the app is split into files.

13. Not Yet

- Router and controller pattern — Session 2.
- Store and seed data — Session 3.
- Validation and status codes — Ali's Session 2.
- Game logic and hints — Ali's Session 4.

Session 2 — Router and Controller Pattern

Learning Objectives

By the end of this session, the team should understand:

- What a router does.
- What a controller does.
- Why they are separate.

1. The Problem

The API has five endpoints. If all logic lived in one file, the file would be huge. Also, the URL patterns and the logic are two different concerns.

2. Familiar Idea

In earlier sessions, we separated concerns: one function for filtering, one for rendering.

The same idea applies to the server.

3. The New Concept

- **Router** answers: "Which controller handles this request?"
- **Controller** answers: "What should happen?"

4. Tiny Example

```
const router = express.Router();
router.get("/", listItems);
router.get("/:id", getItem);
```

5. Project Application

File: `server/routes/mysteries.js`

```
import express from "express";
import {
  getAllMysteries,
  getMysteryById,
  getCluesByMysteryId,
  submitAnswer,
  requestHint,
} from "../controllers/mysteryController.js";

const router = express.Router();

router.get("/", getAllMysteries);
router.get("/:id", getMysteryById);
router.get("/:id/clues", getCluesByMysteryId);
router.post("/:id/answers", submitAnswer);
router.patch("/:id/hint", requestHint);

export default router;
```

6. Code Walkthrough

The router mount

```
app.use("/api/mysteries", mysteriesRouter);
```

What it does: Mounts the router at `/api/mysteries`. **Why it exists:** One mount point, one router file.

What you see: All mystery routes share the prefix.

Each route

```
router.get("/", getAllMysteries);  
router.get("/:id", getMysteryById);  
router.get("/:id/clues", getCluesByMysteryId);  
router.post("/:id/answers", submitAnswer);  
router.patch("/:id/hint", requestHint);
```

What it does: Connects each method+path to a controller function. **Why it exists:** The controller stays focused on logic; the router on wiring. **What you see:** Each endpoint does one thing. **In Plain English:** "When a GET arrives at `/`, run `getAllMysteries`." **Read it as:** "Method, path, handler."

Route order

```
router.get("/:id", getMysteryById);  
router.get("/:id/clues", getCluesByMysteryId);
```

What it does: Registers the specific route after the general one. **Why it exists:** Express matches in order. `/:id/clues` has more segments, so it matches only when the path has `/clues`. **What you see:** Both endpoints work.

7. Common Mistakes

WRONG: Putting logic in the route.

```
router.get("/:id", (req, res) => {  
  const mystery = findMysteryById(Number(req.params.id));  
  // ...  
});
```

The route should stay thin.

WRONG: Forgetting to export the router.

```
const router = express.Router();  
// no export default
```

`app.js` cannot import it.

WRONG: Registering `/:id` before `/:id/clues` if you want clues-specific behavior.

Order matters.

8. Important Decisions

- **Why a separate router file?** The app file stays short; the router is one place for URLs.
- **Why a controller file?** All logic in one place.
- **Why `express.Router()` ?** A mini-app that can be mounted anywhere.

9. Project Flow

```
GET /api/mysteries/1  
  |  
  ▼  
app.js: app.use("/api/mysteries", router)  
  |  
  ▼  
router matches GET /:id  
  |  
  ▼  
getMysteryById(req, res) runs  
  |  
  ▼  
Response sent
```

10. Review Questions

1. What does a router do?
2. What does a controller do?
3. Why separate them?
4. What does `app.use("/api/mysteries", router)` do?
5. Why does route order matter?

11. Glossary

Term	Meaning
Router	A mini Express app that maps paths to handlers.
Controller	A function that handles one request.
Mount	To attach a router at a path prefix.
Handler	A function that processes a request.

12. What You Should Be Able to Explain Now

- What a router and a controller are.
- Why they are separate.
- How the project mounts the router.
- Why route order matters.

13. Not Yet

- Store and seed data — Session 3.
- Validation and status codes — Ali's Session 2.
- Game logic and hints — Ali's Session 4.

Session 3 — Store, Seed Data, and Case 1 Structure

Learning Objectives

By the end of this session, the team should understand:

- Why the project has both seed data and a store.
- What deep-copying does.
- How case 1 is structured in the seed file.

1. The Problem

The game has initial data (two mysteries) and runtime data (which stage the player is on). If we modify the initial data directly, restarting the server would not reset the game.

2. Familiar Idea

In earlier sessions, we copied arrays before modifying them (`.filter` , `spread`). The same idea applies here, at the object level.

3. The New Concept

- **Seed data** is the initial definition. It never changes.
- **Store** is the runtime copy. It changes during play.

Deep copy creates a fully independent copy.

4. Tiny Example

```
const seed = { count: 0 };
const runtime = JSON.parse(JSON.stringify(seed));
runtime.count = 5; // seed.count is still 0
```

5. Project Application

File: `server/store.js`

```
import { initialMysteries } from "../data/mysteries.js";

function deepCopy(obj) {
  return JSON.parse(JSON.stringify(obj));
}

export let mysteries = deepCopy(initialMysteries);

export function resetMysteries() {
  mysteries = deepCopy(initialMysteries);
}

export function findMysteryById(id) {
  return mysteries.find((m) => m.id === id);
}
```

File: `server/data/mysteries.js` (case 1 excerpt)

```

export const initialMysteries = [
  {
    id: 1,
    slug: "sealed-room",
    reveal: "!!.. خلف الباب وجدت صندوقاً نحاسياً ودفتر حسابات باسم أم صالح",
    title: "الغرفة المغلقة في سوق الحميدية",
    intro:
      " + " في عام 1975، أغلق التاجر أبو صالح غرفته في سوق الحميدية بقل غامض",
      " وترك رسالة تقول: من يفتحها يجب أن يمر بثلاثة أفعال. لم يعد أحد يعرف ما يخفيه",
    totalStages: 3,
    stages: [
      {
        id: 0,
        question:
          "القف لا يفتح بالمفتاح. بل بالمعدن الذي صُنِع منه. لونه يشبه الذهب، لكنه ليس ذهباً. ما هو المعدن؟",
        answer: "brass",
        options: ["brass", "iron", "gold", "silver"],
        hint: "B. لونه بين الذهبي والأصفر، واسمه بالإنجليزية يبدأ بحرف",
        clue: "«على القفل نقش صغير: «ثلاث شقوق من معدن أصفر تحرس الباب»",
      },
      {
        id: 1,
        question:
          "تظهر على الشاشة الأرقام التالية: 2، 6، 12، 20، 30، ؟. ما الرقم الذي يجب أن يظهر بعد ذلك؟",
        answer: "42",
        options: ["36", "40", "42", "44"],
        hint: "حاول كتابة كل رقم على شكل حاصل ضرب رقمين متتاليين",
        clue: "«على الشاشة: «كل رقم جديد يولد من الرقم الذي قبله، لكن ليس بالجمع فقط»",
      },
      {
        id: 2,
        question:
          "تظهر على الشاشة الأرقام التالية: 3، 7، 6، 10، 9، 13، ؟. ما الرقم التالي؟",
        answer: "12",
        options: ["11", "12", "14", "16"],
        hint: "لا تحاول إيجاد عملية واحدة لكل الأرقام. راقب ما يحدث بالتناوب",
        clue: "«على الشاشة: «الآلة تتناوب بين خطوتين مختلفتين»",
      },
    ],
    solved: false,
    currentStage: 0,
    hintsUsed: 0,
  },
  // case 2 ...
];

```

6. Code Walkthrough

deepCopy

```
function deepCopy(obj) {  
  return JSON.parse(JSON.stringify(obj));  
}
```

What it does: Creates a fully independent copy. **Why it exists:** Modifying the copy does not affect the seed. **What you see:** Restarting the server resets progress. **In Plain English:** "Serialize and deserialize." **Read it as:** "JSON roundtrip."

export let mysteries

```
export let mysteries = deepCopy(initialMysteries);
```

What it does: Exports a mutable reference to the runtime array. **Why it exists:** Controllers read and mutate this. **What you see:** `mysteries` changes as the game progresses.

resetMysteries

```
export function resetMysteries() {  
  mysteries = deepCopy(initialMysteries);  
}
```

What it does: Replaces the runtime data with a fresh copy. **Why it exists:** The test suite uses this between tests. **What you see:** Tests start from a known state. **Note:** This is not a player feature. It is a test helper.

findMysteryById

```
export function findMysteryById(id) {  
  return mysteries.find((m) => m.id === id);  
}
```

What it does: Returns the matching mystery or `undefined`. **Why it exists:** Every controller needs this lookup.

Case 1 — three stages

What it does: Defines the first mystery. **Why it exists:** The server owns the story and the answers. **What you see:** Three questions in Arabic, each with options, a hint, and a clue.

Note: `answer: "brass"` is a string. In case 2, the answer is an array of aliases. The controller handles both shapes.

7. Common Mistakes

WRONG: Modifying the seed directly.

```
initialMysteries[0].currentStage = 1; // never resets
```

CORRECT: Modify the runtime copy.

WRONG: Shallow copy.

```
const copy = { ...initialMysteries }; // nested objects are still shared
```

CORRECT: Use `JSON.parse(JSON.stringify(...))` for deep copy.

WRONG: Assuming the store persists.

Restarting the server resets everything. This is intentional.

8. Important Decisions

- **Why deep copy?** To fully isolate seed from runtime.
- **Why no database?** To keep the project focused on API and logic, not persistence.
- **Why export `let`?** So the controllers can mutate the array.
- **Why case 1 uses a string answer and case 2 uses an array?** The controller handles both. Case 2 needed aliases for Arabic variants.

9. Project Flow

```
Server starts
  |
  ▼
store.js imports initialMysteries
  |
  ▼
deepCopy creates runtime data
  |
  ▼
Controller mutates runtime data
  |
  ▼
Restart → fresh copy from seed
```

10. Review Questions

1. Why have both seed data and a store?
2. What does `deepCopy` do?
3. Why is there no database?
4. What does `findMysteryById` return?
5. What does `resetMysteries` do?
6. How many stages does case 1 have?

11. Glossary

Term	Meaning
Seed data	Initial data that never changes.
Store	Runtime data that changes.
Deep copy	A copy with no shared references.
In-memory	Stored in the running process.
Stage	One puzzle step in a mystery.

12. What You Should Be Able to Explain Now

- Why seed and store are separate.
- What deep copying does.
- Why the project has no database.
- How `resetMysteries` supports tests.
- How case 1 is structured.

13. Not Yet

- Validation and status codes — Ali's Session 2.
 - Hint budget and public response — Ali's Session 4.
 - Integration tests — Ali's Session 5.
-

PART 2 — ALI MIKDAD

Ali Mikdad — Second Mystery, Validation, and Tests

Session 1 — Second Mystery Data Structure

Learning Objectives

By the end of this session, the team should understand:

- How case 2 differs from case 1.
- Why case 2 uses typed Arabic answers.
- How the answer array works.

1. The Problem

Case 1 uses multiple-choice options. But a puzzle can also accept typed answers — the player writes the answer instead of picking from a list. Case 2 needs this different shape.

2. Familiar Idea

In earlier sessions, a filter accepted one value. But sometimes a value has multiple accepted forms — "10", "١٠", "ten". The same idea applies to answers.

3. The New Concept

- **Options array** — when non-empty, the client shows radio buttons.
- **Answer array** — a list of accepted variants. The server checks each.
- **Empty options** — the client shows a text input.

4. Tiny Example

```
// Case 1: multiple choice
answer: "brass",
options: ["brass", "iron", "gold", "silver"],

// Case 2: typed answer
answer: ["10", "١٠", "ten"],
options: [],
```

5. Project Application

File: `server/data/mysteries.js` (case 2 excerpt)

```

{
  id: 2,
  slug: "arcanus-wizard",
  reveal: "تجتمع الإجابات الثلاث فوق البوابة، وتنطفئ عصا أركانوس",
  title: "مواجهة الساحر أركانوس",
  intro:
    + " «ينقشع الضباب فجأة لتجد نفسك في ساحة قلعة مهجورة أمام الساحر العجوز «أركانوس» يقول لك: أمامي ثلاثة أسئلة، إن أجبت عنها ستفتح لك البوابة السحرية وتمنحك القلعة ثمينة المعرفة الأبدية، وإن خسرت "ستبقى روحك محتجزة هنا للأبد",
  totalStages: 3,
  stages: [
    {
      id: 0,
      question:
        أظهر في كلمة «دقيقة» مرتين، وفي كلمة «قرن» مرة واحدة، ولا أظهر في كلمة «ساعة» ولا في عبارة «ألف»
        "سنة". فمن أكون؟
      answer: ["حرف القاف", "القاف", "قاف", "ق", "حرف ق"],
      options: [],
      hint: "ركّز في طريقة كتابة وهجاء كلمة (دقيقة) وكلمة (قرن)، وابحث عن حرف هجائي يتكرر فيهما",
      clue: "تشتعل العصا السحرية بكلمات من نور: «انس أرقام الوقت والساعات تماماً، الفخ هنا في الكلمات»",
    },
    {
      id: 1,
      question:
        خمسة أشقاء يجلسون في غرفة واحدة؛ الأول يقرأ كتاباً، والثاني يلعب الشطرنج، والثالث يطهو الطعام، والرابع يحل
        "الكلمات المتقاطعة. فماذا يفعل الأخ الخامس؟
      answer: [
        "يلعب الشطرنج",
        "يلعب شطرنج",
        "الشطرنج",
        "شطرنج",
        "يلعب الشطرنج مع الأخ الثاني",
        "يلعب الشطرنج مع أخيه",
        "يلعب مع الأخ الثاني",
      ],
      options: [],
      hint: "الأخ الثاني يلعب الشطرنج، والشطرنج يحتاج دائماً إلى خصم أو منافس معه في الغرفة",
      clue: "«تلمع عينا الساحر ويقول: «انظر إلى الألعاب المذكورة في اللغز، هل يمكن لعبها بشكل فردي؟»",
    },
    {
      id: 2,
      question:
        صندوق خشبي مقفل، يحتوي على ثلاث صناديق حديدية صغيرة. داخل كل صندوق حديدي يوجد صندوقان من
        "الذهب، وداخل كل صندوق ذهب توجد جوهرة واحدة. كم عدد الصناديق الإجمالي في الغرفة؟
      answer: ["صناديق", "عشرة", "عشر صناديق", "عشرة صناديق 10", "10"],
    }
  ]
}

```

```

    options: [],
    hint: "ابداً بالصندوق الخشبي (1)، أضف الصناديق الحديدية (3)، ثم صناديق الذهب (3 = 2 × 3). اجمعها.",
    clue: "تشتد طاقة العصا السحرية: «اقرأ السؤال بدقة شديدة، المطلوب هو حساب الصناديق فقط ولا شأن لك».",
  },
],
solved: false,
currentStage: 0,
hintsUsed: 0,
}

```

File: `client/src/pages/GamePage.tsx` (excerpt)

```

{
  mystery.currentOptions.length > 0 ? (
    <AnswerOptions
      options={mystery.currentOptions}
      selected={selectedAnswer}
      onSelect={setSelectedAnswer}
      disabled={pending !== null}
    />
  ) : (
    <label className="text-answer">
      Your answer
      <input
        value={selectedAnswer}
        onChange={(event) => setSelectedAnswer(event.target.value)}
        dir="auto"
        maxLength={200}
        required
        disabled={pending !== null}
      />
    </label>
  );
}

```

6. Code Walkthrough

The empty options array

```
options: [],
```

What it does: Tells the client there are no predefined choices. **Why it exists:** Case 2 is a typed-answer puzzle. **What you see:** A text input instead of radio buttons. **In Plain English:** "No options means let the player type." **Read it as:** "Length check decides input type."

The answer array

```
answer: ["حرف القاف", "القاف", "قاف", "ق", "حرف ق"],
```

What it does: Lists accepted variants. **Why it exists:** Arabic answers can be written many ways. **What you see:** All listed forms are accepted. **In Plain English:** "Accept these specific spellings." **Read it as:** "Aliases for one solution."

The client branch

```
{mystery.currentOptions.length > 0 ? (
  <AnswerOptions ... />
) : (
  <label className="text-answer"> ... </label>
)}
```

What it does: Chooses the input type based on the data. **Why it exists:** One page handles both puzzle styles. **What you see:** Radio buttons for case 1, text input for case 2. **In Plain English:** "If options exist, show them; otherwise, show text input."

7. Common Mistakes

WRONG: Hardcoding case 2's text input.

```
{
  id ≡ "2" ? <input /> : <AnswerOptions />;
}
```

CORRECT: Check `currentOptions.length`. The data decides.

WRONG: Forgetting to normalize Arabic input.

"قاف" and " قاف " (with trailing space) should both work. The controller trims.

WRONG: Adding every possible spelling.

List only the variants you are willing to accept. Do not try to cover every Arabic dialect.

8. Important Decisions

- **Why empty options instead of a special flag?** The client already checks length. No new field needed.

- **Why an array of answers?** Arabic variants are common. This makes acceptance explicit.
- **Why list only a few aliases?** The test file verifies each one. Adding more means adding tests.

9. Project Flow

```
GET /api/mysteries/2
|
▼
toPublicMystery returns currentOptions: []
|
▼
GamePage checks length === 0
|
▼
Renders the text input
|
▼
Player types "قاف" → selectedAnswer state
|
▼
Submit → POST /api/mysteries/2/answers
```

10. Review Questions

1. How does case 2 differ from case 1?
2. Why does case 2 use an answer array?
3. What does the client check to decide the input type?
4. Why list only a few aliases?

11. Glossary

Term	Meaning
Options array	The list of radio choices. Empty means no choices.
Alias	An explicitly accepted alternative answer.
Text input	A free-form field for typed answers.
Multiple choice	A puzzle style with predefined options.

12. What You Should Be Able to Explain Now

- How case 2 differs from case 1.
- Why case 2 uses an answer array.

- How the client decides which input to show.
- Why aliases are explicit.

13. Not Yet

- Answer validation — Session 2.
 - Normalization — Session 3.
 - Hint budget — Session 4.
 - Tests — Session 5.
-

Session 2 — Answer Validation and HTTP Status Codes

Learning Objectives

By the end of this session, the team should understand:

- Why the server must validate input.
- What each status code means.
- How the controller validates and responds.

1. The Problem

The client can send anything. Even if the browser restricts input, a script could bypass it. The server must validate.

2. Familiar Idea

In earlier sessions, we checked `if (task.completed)` before counting. The same idea applies to request bodies.

3. The New Concept

- **Validation** checks input before using it.
- **Status codes** tell the client how the request was handled.

The project uses:

- **200** — success.
- **400** — invalid input.
- **404** — resource not found.
- **409** — conflict with current state.
- **413** — payload too large.

4. Tiny Example

```
if (!req.body || typeof req.body.answer !== "string") {  
  return res.status(400).json({ error: "Answer must be a string." });  
}
```

5. Project Application

File: `server/controllers/mysteryController.js` (excerpt)

```
export function submitAnswer(req, res) {
  const id = Number(req.params.id);
  const mystery = findMysteryById(id);

  if (!mystery) {
    return res.status(404).json({ error: `No mystery found with id ${id}.` });
  }

  if (
    !req.body ||
    typeof req.body.answer !== "string" ||
    req.body.answer.trim() === "" ||
    req.body.answer.length > 200
  ) {
    return res
      .status(400)
      .json({
        error: "Answer must be a non-empty string of at most 200 characters.",
      });
  }

  if (mystery.solved) {
    return res.status(409).json({ error: "This mystery is already solved." });
  }

  const currentStage = mystery.stages[mystery.currentStage];
  const submitted = req.body.answer.trim().toLowerCase();
  const acceptedAnswers = Array.isArray(currentStage.answer)
    ? currentStage.answer
    : [currentStage.answer];
  const isCorrect = acceptedAnswers.some(
    (answer) => answer.trim().toLowerCase() === submitted,
  );

  if (!isCorrect) {
    return res.json({
      correct: false,
      message: "That does not match the evidence. Try again.",
    });
  }

  const nextStage = mystery.currentStage + 1;

  if (nextStage ≥ mystery.totalStages) {
    mystery.solved = true;
    return res.json({
```

```
        correct: true,
        message: "The final lock clicks open. The mystery is solved!",
        nextStage: null,
    });
}

mystery.currentStage = nextStage;
res.json({
    correct: true,
    message: "The lock clicks open. You may continue.",
    nextStage: nextStage,
});
}
```

6. Code Walkthrough

Number conversion

```
const id = Number(req.params.id);
```

What it does: Converts the URL parameter from string to number. **Why it exists:** `req.params.id` is always a string. **What you see:** `findMysteryById` compares numbers correctly.

404 — missing mystery

```
if (!mystery) {
    return res.status(404).json({ error: `No mystery found with id ${id}.` });
}
```

What it does: Returns 404 if the ID does not match. **Why it exists:** A missing resource deserves a specific status. **What you see:** The client shows "No mystery found with id 999."

400 — invalid input

```
if (
  !req.body ||
  typeof req.body.answer !== "string" ||
  req.body.answer.trim() === "" ||
  req.body.answer.length > 200
) {
  return res
    .status(400)
    .json({
      error: "Answer must be a non-empty string of at most 200 characters.",
    });
}
```

What it does: Rejects missing, non-string, empty, or over-long answers. **Why it exists:** The server should not trust the client. **What you see:** Clear error.

409 — conflict

```
if (mystery.solved) {
  return res.status(409).json({ error: "This mystery is already solved." });
}
```

What it does: Rejects answers to a solved mystery. **Why it exists:** The action conflicts with the current state. **What you see:** Clear error.

Wrong answer — 200 with `correct: false`

```
if (!isCorrect) {
  return res.json({
    correct: false,
    message: "That does not match the evidence. Try again.",
  });
}
```

What it does: Returns a successful response with `correct: false`. **Why it exists:** A wrong guess is a valid puzzle outcome, not a protocol error. **What you see:** "That does not match the evidence."

Correct answer — advance or solve

```
const nextStage = mystery.currentStage + 1;
if (nextStage ≥ mystery.totalStages) {
  mystery.solved = true;
  return res.json({ correct: true, message: "...", nextStage: null });
}
mystery.currentStage = nextStage;
res.json({ correct: true, message: "...", nextStage });
```

What it does: Advances the stage or marks the mystery solved. **Why it exists:** The server owns progress.

What you see: The next stage, or the final reveal.

7. Common Mistakes

WRONG: Trusting the client.

```
const answer = req.body.answer; // may be undefined
```

CORRECT: Validate first.

WRONG: Using 200 for errors.

```
return res.json({ error: "Not found" }); // status is still 200
```

CORRECT:

```
return res.status(404).json({ error: "... " });
```

WRONG: Sending the accepted answers.

The `toPublicMystery` function excludes them.

8. Important Decisions

- **Why 400 for invalid input?** The client sent something wrong.
- **Why 404 for missing mystery?** The resource does not exist.
- **Why 409 for already-solved?** Conflict with current state.
- **Why 200 for a wrong guess?** It is a valid puzzle outcome.

9. Project Flow

```
POST /api/mysteries/1/answers { answer: "brass" }
  |
  ▼
Controller: findMysteryById(1) → found
  |
  ▼
Validate body → passed
  |
  ▼
Not solved → continue
  |
  ▼
Normalize "brass" → matches acceptedAnswers
  |
  ▼
nextStage = 1 < 3
  |
  ▼
mystery.currentStage = 1
  |
  ▼
Response: { correct: true, message: "...", nextStage: 1 }
```

10. Review Questions

1. Why must the server validate input?
2. What does each status code mean here?
3. Why is a wrong guess returned as 200?
4. Why does `findMysteryById` need a number?

11. Glossary

Term	Meaning
Validation	Checking input before using it.
Status code	A number describing how a request was handled.
Conflict	An action incompatible with the current state.
Guard clause	An early check that returns before further work.

12. What You Should Be Able to Explain Now

- Why the server validates.
- What each status code signals.
- Why the server owns progression.

13. Not Yet

- Normalization and aliases — Session 3.
 - Hint budget — Session 4.
 - Tests — Session 5.
-

Session 3 — Normalization and Aliases

Learning Objectives

By the end of this session, the team should understand:

- What normalization is.
- Why it matters for Arabic and numeric answers.
- How the project normalizes.

1. The Problem

A player types `" BRASS "` or `"Brass"` or `"brass"`. The answer is the same. Without normalization, the server would reject valid input.

For case 2, the same puzzle can be written `"10"`, `"١٠"`, or `"عشرة"`. Each is a correct answer, but each is a different string.

2. Familiar Idea

In earlier sessions, we lowercased search text so `"JAVA"` matched `"java"`. The same idea applies here.

3. The New Concept

- **Normalization** is a small, deliberate transformation before comparison.
- **Alias** is an explicitly accepted variant.

The project uses:

- `trim()` — removes surrounding spaces.
- `toLowerCase()` — makes case-insensitive comparisons.
- An array of aliases for each stage.

4. Tiny Example

```
const submitted = " BRASS ".trim().toLowerCase(); // "brass"
const accepted = "brass".trim().toLowerCase(); // "brass"
submitted === accepted; // true
```

5. Project Application

File: `server/controllers/mysteryController.js` (excerpt)

```
const submitted = req.body.answer.trim().toLowerCase();
const acceptedAnswers = Array.isArray(currentStage.answer)
  ? currentStage.answer
  : [currentStage.answer];
const isCorrect = acceptedAnswers.some(
  (answer) => answer.trim().toLowerCase() === submitted,
);
```

6. Code Walkthrough

Normalize the submitted answer

```
const submitted = req.body.answer.trim().toLowerCase();
```

What it does: Trims spaces and lowercases. **Why it exists:** The player should not need to match case or spacing exactly. **What you see:** " BRASS " and "brass" both become "brass". **In Plain English:** "Make the input consistent."

Normalize the accepted answer

```
const acceptedAnswers = Array.isArray(currentStage.answer)
  ? currentStage.answer
  : [currentStage.answer];
const isCorrect = acceptedAnswers.some(
  (answer) => answer.trim().toLowerCase() === submitted,
);
```

What it does: Wraps a single answer in an array; compares each alias after normalization. **Why it exists:** Case 1 stores one answer; case 2 stores many. **What you see:** Both puzzle types work with the same code. **In Plain English:** "Check every alias; accept if any matches."

Note on Arabic: `toLowerCase()` does not affect Arabic letters. For Arabic, the alias list itself must include the variants. That is why case 2 lists "ق", "القاف", "قَاف", etc.

7. Common Mistakes

WRONG: Comparing without normalization.

```
if (req.body.answer === currentStage.answer) { ... }
```

" BRASS " would fail.

WRONG: Only handling one answer shape.

```
const isCorrect = submitted === currentStage.answer.toLowerCase();
```

This fails when `answer` is an array.

WRONG: Trying to handle every Arabic variant in code.

There is no built-in normalization for Arabic. Aliases must be listed in the data.

WRONG: Accepting partial matches.

`includes` would accept `"brass"` when the answer is `"brassiere"`. The project uses `===`, not `includes`.

8. Important Decisions

- **Why `trim()` and `toLowerCase()` ?** Simple, predictable normalization.
- **Why array aliases?** Arabic variants are common; listing them is honest.
- **Why `===` and not `includes` ?** To avoid accepting unintended partial matches.
- **Why not fuzzy matching?** Scope. Fuzzy matching is a much bigger topic.

9. Project Flow

```

Player types " BRASS "
    |
    ▼
req.body.answer = " BRASS "
    |
    ▼
submitted = "brass" (after trim + lowercase)
    |
    ▼
acceptedAnswers = ["brass"]
    |
    ▼
some(a ⇒ a ≡ "brass") → true
    |
    ▼
Stage advances
  
```

10. Review Questions

1. What is normalization?
2. Why trim and lowercase?
3. Why does case 1 use one answer and case 2 use many?
4. Why not use `includes` ?
5. Why not add every possible Arabic spelling?

11. Glossary

Term	Meaning
Normalization	A defined transformation before comparison.
Alias	An explicitly accepted variant.
Partial match	A match on part of a string; not used here.
Fuzzy match	Approximate matching; out of scope.

12. What You Should Be Able to Explain Now

- Why normalization matters.
- How the project normalizes.
- Why aliases are explicit.
- Why `===` is used instead of `includes` .

13. Not Yet

- Hint budget — Session 4.
 - Tests — Session 5.
-

Session 4 — Hint Budget and Public Response

Learning Objectives

By the end of this session, the team should understand:

- How the hint budget works.
- Why `hintsRemaining` is computed.
- How `toPublicMystery` protects secrets.

1. The Problem

The game allows three hints per mystery. The server must enforce this, not the client. Also, the client must not receive the accepted answer or the reveal before it is solved.

2. Familiar Idea

In earlier sessions, we counted completed tasks. The same idea applies here: the server counts hints used.

3. The New Concept

- `hintsUsed` — number of hints requested.
- `hintsRemaining` — computed as `Math.max(0, 3 - hintsUsed)`.
- `toPublicMystery` — builds the safe shape sent to the client.

4. Tiny Example

```
if (mystery.hintsUsed ≥ 3) {  
  return res.status(409).json({ error: "No hints remaining." });  
}  
mystery.hintsUsed += 1;
```

5. Project Application

File: `server/controllers/mysteryController.js` (excerpt)

```
export function requestHint(req, res) {
  const id = Number(req.params.id);
  const mystery = findMysteryById(id);

  if (!mystery) {
    return res.status(404).json({ error: `No mystery found with id ${id}.` });
  }

  if (mystery.solved) {
    return res.status(409).json({ error: "This mystery is already solved." });
  }

  if (mystery.hintsUsed ≥ 3) {
    return res
      .status(409)
      .json({ error: "No hints remaining for this mystery." });
  }

  const currentStage = mystery.stages[mystery.currentStage];
  mystery.hintsUsed += 1;

  const hintsRemaining = Math.max(0, 3 - mystery.hintsUsed);

  res.json({
    hint: currentStage.hint,
    hintsRemaining: hintsRemaining,
  });
}
```

File: `server/controllers/mysteryController.js` (excerpt)

```
export function toPublicMystery(mystery) {
  const currentStage = mystery.solved
    ? null
    : mystery.stages[mystery.currentStage];

  return {
    id: mystery.id,
    slug: mystery.slug,
    title: mystery.title,
    intro: mystery.intro,
    totalStages: mystery.totalStages,
    currentStage: mystery.currentStage,
    solved: mystery.solved,
    hintsUsed: mystery.hintsUsed,
    hintsRemaining: Math.max(0, 3 - mystery.hintsUsed),
    reveal: mystery.solved ? mystery.reveal : null,
    currentQuestion: currentStage ? currentStage.question : "",
    currentOptions: currentStage ? currentStage.options : [],
  };
}
```

6. Code Walkthrough

requestHint — check solved first

```
if (mystery.solved) {
  return res.status(409).json({ error: "This mystery is already solved." });
}
```

What it does: Rejects hints on a solved mystery. **Why it exists:** The game is over; hints are irrelevant.

What you see: Clear error.

Check budget

```
if (mystery.hintsUsed ≥ 3) {
  return res
    .status(409)
    .json({ error: "No hints remaining for this mystery." });
}
mystery.hintsUsed += 1;
```

What it does: Rejects if all hints used; otherwise increments. **Why it exists:** The game allows exactly three hints. **What you see:** The frontend's hint button disables at 0 remaining.

hintsRemaining

```
const hintsRemaining = Math.max(0, 3 - mystery.hintsUsed);
```

What it does: Computes the remaining budget. **Why it exists:** The client displays it. **What you see:** "Hints remaining: 2". **In Plain English:** "Never show a negative number."

toPublicMystery

```
export function toPublicMystery(mystery) {
  const currentStage = mystery.solved ? null :
mystery.stages[mystery.currentStage];
  return {
    ...
    hintsRemaining: Math.max(0, 3 - mystery.hintsUsed),
    reveal: mystery.solved ? mystery.reveal : null,
    currentQuestion: currentStage ? currentStage.question : "",
    currentOptions: currentStage ? currentStage.options : [],
  };
}
```

What it does: Builds the public shape; excludes the accepted answer. **Why it exists:** The client should not receive secret data. **What you see:** After solving, `reveal` contains the ending text. **In Plain English:** "Send only what the client needs." **Read it as:** "Explicit fields, no `stages`."

Key exclusions:

- `stages` — includes `answer`, `hint`, `clue`. Not sent.
- `reveal` — sent only when `solved: true`.
- `answer` — never sent.

7. Common Mistakes

WRONG: Forgetting `Math.max`.

`3 - 4` would be `-1`, which displays incorrectly.

WRONG: Sending the accepted answer in the public response.

`toPublicMystery` excludes `stages`.

WRONG: Letting the client decide the budget.

The client can only display `hintsRemaining`. The server enforces.

WRONG: Sending `reveal` before solving.

`reveal: mystery.solved ? mystery.reveal : null` prevents this.

8. Important Decisions

- **Why three hints?** A simple, fair budget.
- **Why `hintsRemaining` computed, not stored?** Less state to keep consistent.
- **Why `reveal` is `null` until solved?** A spoiler must not leak.
- **Why `toPublicMystery` instead of sending the raw object?** Security and shape control.

9. Project Flow

```
graph TD
    A[Player clicks "Request hint"] --> B[PATCH /api/mysteries/1/hint]
    B --> C[Controller: find mystery, check solved, check budget]
    C --> D[Increment hintsUsed → 1]
    D --> E[Compute hintsRemaining → 2]
    E --> F[Return { hint: "...", hintsRemaining: 2 }]
    F --> G[Frontend shows hint, updates budget display]
    G --> H[Player submits final correct answer]
    H --> I[Controller: solved = true, return reveal]
    I --> J[Frontend navigates to /result/1]
```

10. Review Questions

1. What does `hintsUsed` track?
2. Why is `reveal` null until solved?
3. Why does `toPublicMystery` exclude `stages` ?
4. Why is `hintsRemaining` computed instead of stored?

5. Why does the server enforce the hint budget?

11. Glossary

Term	Meaning
<code>hintsUsed</code>	Number of hints requested.
<code>hintsRemaining</code>	Computed as <code>Math.max(0, 3 - hintsUsed)</code> .
<code>reveal</code>	The ending text, only exposed after solving.
Public shape	The data the server chooses to expose.
Secret data	Answers, hints, and clues not sent to the client.

12. What You Should Be Able to Explain Now

- How the hint budget works.
- How `toPublicMystery` protects secrets.
- Why `reveal` is null until solved.
- Why `hintsRemaining` is computed.

13. Not Yet

- Integration tests — Session 5.

Session 5 — Integration Tests

Learning Objectives

By the end of this session, the team should understand:

- What an integration test is.
- Why the project uses `node:test`.
- How tests verify server behavior.

1. The Problem

How do we know the server works across all cases? Manual testing with `curl` is slow. A test suite catches regressions.

2. Familiar Idea

In earlier sessions, we manually checked that the page worked. The same idea applies here, but automated.

3. The New Concept

An **integration test** sends real HTTP requests to the app and checks the response. It exercises parsing, routing, controllers, and the store together.

The project uses Node's built-in `node:test` runner.

4. Tiny Example

```
import assert from "node:assert/strict";
import { test } from "node:test";

test("1 + 1 is 2", () => {
  assert.equal(1 + 1, 2);
});
```

5. Project Application

File: `server/tests/mysteries.test.js` (excerpts)

```
import assert from "node:assert/strict";
import { after, before, beforeEach, test } from "node:test";
import app from "../app.js";
import { initialMysteries } from "../data/mysteries.js";
import { resetMysteries, findMysteryById } from "../store.js";

let server;
let baseUrl;

before(async () => {
  server = await new Promise((resolve) => {
    const listener = app.listen(0, "127.0.0.1", () => resolve(listener));
  });
  baseUrl = `http://127.0.0.1:${server.address().port}/api/mysteries`;
});

after(
  () =>
    new Promise((resolve, reject) => {
      server.close((error) => (error ? reject(error) : resolve()));
    }),
);

beforeEach(() => resetMysteries());

async function request(path = "", method = "GET", body) {
  const response = await fetch(`${baseUrl}${path}`, {
    method,
    headers: { "Content-Type": "application/json" },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  return { status: response.status, data: await response.json() };
}

test("both mysteries are listed without exposing accepted answers", async () => {
  const { status, data } = await request();
  assert.equal(status, 200);
  assert.deepEqual(
    data.map(({ id }) => id),
    [1, 2],
  );
  for (const mystery of data) {
    assert.equal("stages" in mystery, false);
    assert.equal("answer" in mystery, false);
    assert.equal(mystery.reveal, null);
  }
});
```

```

    assert.equal(mystery.hintsRemaining, 3);
    const details = await request(`/${mystery.id}`);
    assert.deepEqual(details.data, mystery);
    const clues = await request(`/${mystery.id}/clues`);
    assert.equal(clues.status, 200);
    assert.equal(clues.data.length, mystery.totalStages);
    assert.ok(clues.data.every((clue) => typeof clue.text === "string"));
  }
});

test("both multiple-choice and free-text mysteries can be completed", async () => {
  for (const [id, answers] of [
    [1, ["BRASS ", "42", "12"]],
    [2, ["١٠", "يلعب الشطرنج", "قاف"]],
  ]) {
    for (let stage = 0; stage < answers.length; stage += 1) {
      const wrong = await request(`/${id}/answers`, "POST", {
        answer: "wrong",
      });
      assert.equal(wrong.status, 200);
      assert.equal(wrong.data.correct, false);
      assert.equal((await request(`/${id}`)).data.currentStage, stage);
      const result = await request(`/${id}/answers`, "POST", {
        answer: answers[stage],
      });
      assert.equal(result.status, 200);
      assert.equal(result.data.correct, true);
      assert.equal(result.data.nextStage, stage === 2 ? null : stage + 1);
    }
    const completed = (await request(`/${id}`)).data;
    assert.equal(completed.solved, true);
    assert.ok(completed.reveal.length > 50);
    assert.equal(completed.currentQuestion, "");
    assert.deepEqual(completed.currentOptions, []);
    assert.equal(
      (await request(`/${id}/answers`, "POST", { answer: "10" })).status,
      409,
    );
    assert.equal((await request(`/${id}/hint`, "PATCH")).status, 409);
  }
});

test("every supplied free-text alias is accepted", async () => {
  for (const [stageIndex, stage] of initialMysteries[1].stages.entries()) {
    for (const answer of stage.answer) {
      resetMysteries();
    }
  }
}

```

```

        findMysteryById(2).currentStage = stageIndex;
        const result = await request("/2/answers", "POST", { answer });
        assert.equal(result.status, 200);
        assert.equal(result.data.correct, true, answer);
    }
}
});

test("invalid answers and missing mysteries return the expected errors", async () => {
    for (const body of [
        undefined,
        {},
        [],
        { answer: "" },
        { answer: "  " },
        { answer: 10 },
        { answer: ["ق"] },
        { answer: "a".repeat(201) },
    ]) {
        assert.equal((await request("/2/answers", "POST", body)).status, 400);
        assert.equal(findMysteryById(2).currentStage, 0);
    }
    for (const [path, method] of [
        ["", "GET"],
        ["/clues", "GET"],
        ["/answers", "POST"],
        ["/hint", "PATCH"],
    ]) {
        assert.equal((await request(`/999${path}`, method)).status, 404);
    }
});

test("malformed JSON and unknown routes produce readable errors", async () => {
    const response = await fetch(`${baseUrl}/1/answers`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: '{"answer":',
    });
    assert.equal(response.status, 400);
    assert.match(await response.json().error, /valid JSON/);
    assert.equal((await request("/1/unknown")).status, 404);
    assert.equal(findMysteryById(1).currentStage, 0);
});

test("the three-hint budget is enforced separately for each mystery", async ()
=> {

```

```
for (const id of [1, 2]) {
  for (const remaining of [2, 1, 0]) {
    const result = await request(`/ ${id}/hint`, "PATCH");
    assert.equal(result.status, 200);
    assert.equal(result.data.hintsRemaining, remaining);
    assert.ok(result.data.hint);
  }
  assert.equal((await request(`/ ${id}/hint`, "PATCH")).status, 409);
  assert.equal((await request(`/ ${id}`)).data.hintsUsed, 3);
}
});
```

6. Code Walkthrough

Starting an isolated server

```
before(async () => {
  server = await new Promise((resolve) => {
    const listener = app.listen(0, "127.0.0.1", () => resolve(listener));
  });
  baseUrl = `http://127.0.0.1:${server.address().port}/api/mysteries`;
});
```

What it does: Starts the app on an ephemeral port. **Why it exists:** The test runs against a real HTTP server, not a mock. **What you see:** `baseUrl` includes the assigned port. **In Plain English:** "Start the app on any free port."

Resetting between tests

```
beforeEach(() => resetMysteries());
```

What it does: Restores the runtime store before each test. **Why it exists:** Tests should not affect each other. **What you see:** Each test starts from a known state.

The `request` helper

```
async function request(path = "", method = "GET", body) {
  const response = await fetch(`${baseUrl}${path}`, {
    method,
    headers: { "Content-Type": "application/json" },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  return { status: response.status, data: await response.json() };
}
```

What it does: Sends a request and returns status and JSON. **Why it exists:** Avoids repeating `fetch` boilerplate. **What you see:** Tests read like short assertions.

Test 1 — privacy and reveals

```
test("both mysteries are listed without exposing accepted answers", async () => {
  const { status, data } = await request();
  assert.equal(status, 200);
  assert.deepEqual(data.map(({ id }) => id), [1, 2]);
  for (const mystery of data) {
    assert.equal("stages" in mystery, false);
    assert.equal("answer" in mystery, false);
    assert.equal(mystery.reveal, null);
    assert.equal(mystery.hintsRemaining, 3);
    ...
  }
});
```

What it does: Verifies that the public response does not expose secrets. **Why it exists:** Security check.

What you see: `stages` and `answer` are absent; `reveal` is null.

Test 2 — full playthrough

What it does: Plays both cases from start to finish. **Why it exists:** The most important behavior. **What you see:** A wrong guess returns 200 with `correct: false`; the stage does not advance.

Test 3 — every alias

What it does: Tries every listed alias for case 2. **Why it exists:** Confirms the alias list is complete and correct.

Test 4 — invalid input

What it does: Tries many malformed bodies and missing mysteries. **Why it exists:** Verifies 400 and 404.

What you see: Each body returns 400; the stage stays at 0.

Test 5 — malformed JSON and unknown routes

What it does: Sends broken JSON and an unknown path. **Why it exists:** Confirms error middleware and 404 handler.

Test 6 — hint budget

What it does: Requests hints until exhaustion for each mystery. **Why it exists:** Confirms the budget is per-mystery and enforced.

7. Common Mistakes

WRONG: Sharing state between tests.

Without `resetMysteries()`, one test's progress affects the next.

WRONG: Mocking the store.

Integration tests use the real app. Mocking would defeat the purpose.

WRONG: Forgetting `assert.deepEqual`.

`assert.equal` would compare object references, not values.

WRONG: Assuming `assert` is available.

Import it from `node:assert/strict`.

8. Important Decisions

- **Why `node:test`?** Built-in, no extra dependency.
- **Why an ephemeral port?** Avoids conflicts with a running dev server.
- **Why reset between tests?** Isolation.
- **Why test both cases?** Confirms the same code handles both shapes.

9. Project Flow

```
npm test in server/  
  |  
  ▼  
node --test runs mysteries.test.js  
  |  
  ▼  
before: start isolated server  
  |  
  ▼  
beforeEach: reset store  
  |  
  ▼  
test: send HTTP requests, assert responses  
  |  
  ▼  
after: close the server
```

10. Review Questions

1. What is an integration test?
2. Why start a real server instead of mocking?
3. Why reset the store between tests?
4. Why use an ephemeral port?
5. What does the alias test verify?

11. Glossary

Term	Meaning
Integration test	A test that exercises connected parts together.
Test isolation	Keeping test state separate.
Ephemeral port	A port assigned by the OS on the fly.
Assertion	A check that must be true.
Test runner	The tool that runs tests.

12. What You Should Be Able to Explain Now

- What an integration test is.
- Why `node:test` is used.

- How tests reset state.
- What each test verifies.

13. Not Yet

- Frontend tests — not part of this project.
 - Database tests — no database exists.
 - CI/CD — out of scope.
-

End of file.

This covers the full backend, split by ownership:

- **Hamid Al Haj** — Sessions 1–3: Node/Express foundations, router/controller pattern, store/seed/case 1.
- **Ali Mikdad** — Sessions 1–5: case 2 data, validation, normalization, hint budget/public response, integration tests.